Database Sharding Fundamentals

choosing a shard key, and the pitfalls

Watch the 3-min video

In one line: sharding splits rows across many databases so writes and storage scale, not just reads. Everything depends on the shard key: high cardinality, even spread, and present in most queries. It brings real costs, so shard last, not first.

1 The problem: one machine is not enough

Your users table has **2 billion rows** and **40 TB** of data. The primary is on the biggest machine you can buy, and its disks are at their write limit. Read replicas do not help, because every replica must still apply every write. It is time to shard.

2 What sharding is

Sharding means horizontal partitioning **across machines**. Instead of one huge database, you run many smaller ones, called **shards**, and each holds a subset of the rows. Together they scale **writes and storage**, not just reads. Each shard is usually replicated too, for availability.

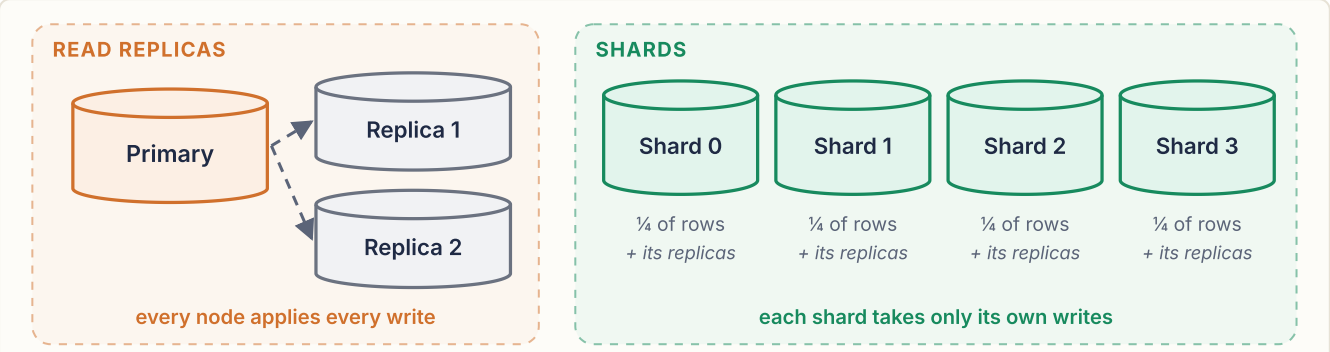


Figure 1. Replicas copy everything, so writes do not scale. Shards split the rows, so each machine takes only its share of writes and storage.

3 The shard key decides everything

The **shard key** is the column that decides which shard owns a row. A good shard key:

- has **high cardinality** (many distinct values), so data can be split finely;
- **spreads data and traffic evenly**, with no value much hotter than the rest;
- **appears in most queries**, so each query can go to exactly one shard.

Bad choices: `country`, because a few countries dominate, and a creation timestamp, because every new row lands on the newest shard.

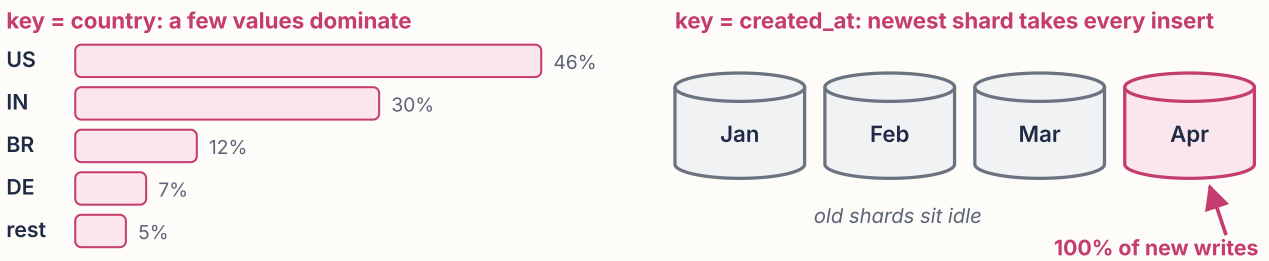


Figure 2. Low cardinality gives skew (the US shard is overloaded); a monotonically increasing key gives a write hotspot on the newest shard.

4 Example: shard users by user ID

Hash the user ID and pick one of, say, four shards: `shard = hash(user_id) mod 4`. Then store each user's **orders, addresses and settings** by the same user ID, so they live on the same shard. Loading a user's account touches **exactly one shard**, and a transaction on one user's data stays local.

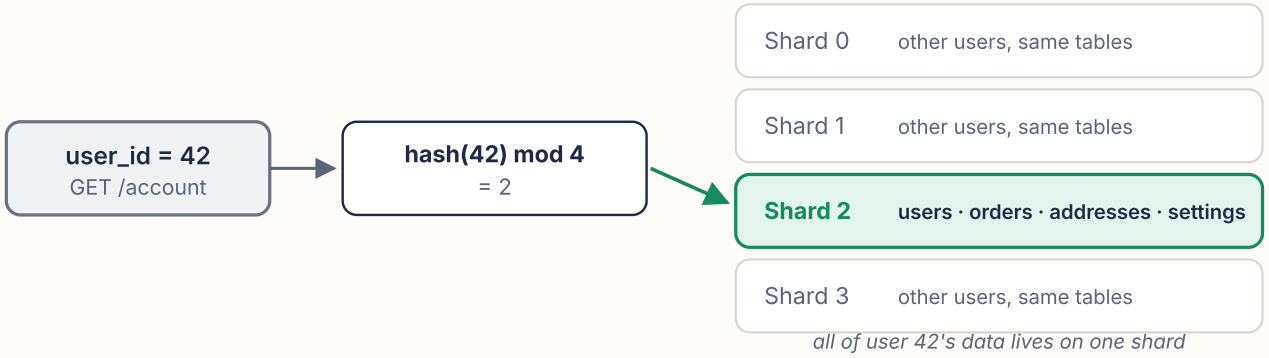


Figure 3. Co-location: every table keyed by user_id is sharded the same way, so one user's data never spans shards.

WORKED EXAMPLE: THE NUMBERS

- **Four shards:** 2 B rows / 4 = **500 M rows** and 40 TB / 4 = **10 TB** per shard. Each primary takes about a quarter of the writes.
- **Room to grow:** at 16 shards it is 125 M rows and 2.5 TB each, a comfortable size for one machine.
- **Why plain mod N is painful:** going from 4 to 5 shards changes `hash mod N` for about **80%** of users, so most data must move. That is why real systems use many small virtual shards or consistent hashing (next lesson).

5 Routing queries to the right shard

Something has to route each query:

- **A library in the application** computes the shard itself.
- **A proxy layer**, like **Vitess** for MySQL, or a **coordinator**, like **Citus** for Postgres.
- **A lookup service**: a directory mapping each key to a shard. Flexible (move one tenant at a time), but it adds a hop and must itself be highly available.
- **Distributed SQL** databases, like **Spanner** and **CockroachDB**, shard and rebalance automatically.

6 The problems sharding brings, and when to do it

- **Queries without the shard key**, like find a user by email, must ask every shard: a **scatter-gather**. The usual fix is a lookup table from email to user ID.

- **Cross-shard transactions** need two-phase commit or sagas.
- **Cross-shard joins** are expensive; co-locate data that is joined together.
- **Global uniqueness:** unique constraints and auto-increment IDs no longer work across shards, so switch to **UUIDs** or **Snowflake IDs** (time + machine + sequence, 64 bits).
- **Rebalancing**, moving data between shards, is hard, and operations get much more complex.

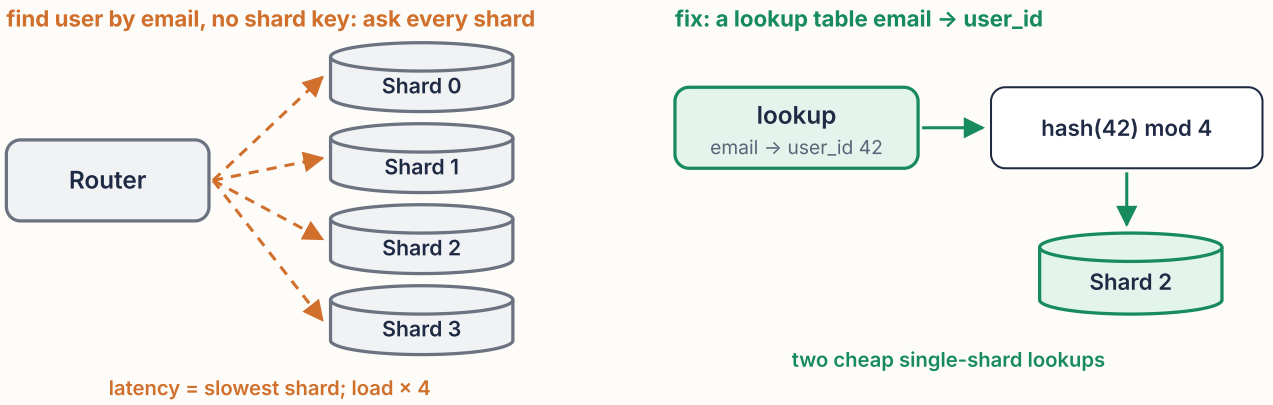


Figure 4. A query without the shard key fans out to every shard. A small lookup table turns it into two single-shard reads.

SHARD LAST, NOT FIRST

Before sharding: optimize queries and indexes, add caching and read replicas, scale up the machine, partition big tables inside one database, and archive old data. Many companies run very large businesses on a single, well-tuned primary.

COMMON MISTAKES

- Sharding before exhausting indexes, caching, replicas and a bigger machine.
- Choosing a low-cardinality or time-based shard key.
- Sharding related tables by different keys, so every page needs a cross-shard join.
- Keeping auto-increment IDs, which collide across shards.

7 Key terms

Shard: one database holding a subset of rows

Hotspot: one shard taking far more load

Lookup table: maps a secondary key to the shard key

Shard key: the column that picks the shard

Co-location: related rows on the same shard

Snowflake ID: 64-bit time-ordered unique ID

Cardinality: number of distinct key values

Scatter-gather: a query sent to every shard

Rebalancing: moving data between shards

8 Interview questions

Q1. How do you choose a shard key? BASIC

Pick a high-cardinality key that spreads data and load evenly, is not monotonically increasing, and matches the main access pattern, so most queries hit one shard. For a user-centric app, user_id is the classic choice.

Q2. What problems does sharding introduce? BASIC

Cross-shard queries and joins, distributed transactions (2PC or sagas), global uniqueness and IDs, rebalancing, and much more operational complexity.

Q3. Why don't read replicas solve a write bottleneck? BASIC

Every replica must apply every write, so write load per node stays the same. Sharding splits the writes: with 4 shards, each primary takes about a quarter.

Q4. Shard an orders system. Key: order_id, user_id or created_at? SCENARIO

user_id, if most reads are “this user's orders”: one shard per request, and orders co-locate with the user. created_at makes the newest shard a write hotspot. order_id spreads well but scatters a user's orders across every shard.

Q5. Login looks users up by email, but you shard by user_id. What do you do? SCENARIO

Keep a lookup table email → user_id (itself sharded by email, or in a fast key-value store). Login does two single-shard reads instead of a scatter-gather across every shard.

Q6. A 2 TB database's writes are near the disk limit. Should you shard? SCENARIO

Probably not yet. Check slow queries and missing indexes, batch writes, move hot data to faster disks or a bigger instance, partition and archive old data. 2 TB fits on one machine; shard when those options are exhausted.

Q7. You shard by hash(user_id) mod 4. Six months later you need 5 shards. What happens? TRICKY

hash mod 5 differs from hash mod 4 for about 80% of keys, so most rows must move. Plan ahead with many virtual shards (e.g. 1,024 mapped onto 4 machines) or consistent hashing, so only a small share moves.

Q8. Why can't you keep auto-increment primary keys after sharding? TRICKY

Each shard has its own counter, so shard 0 and shard 1 both create ID 1001. Use UUIDs, Snowflake-style IDs (timestamp + machine ID + sequence), or per-shard ranges or offsets.

Next: SD-E-023 · Range vs Hash Partitioning backendroadmap.com